

infer – A Notation and System for Inferring Event Stream Abstractions

Sean Kauffman¹

`sean.kauffman@uwaterloo.ca`

Rajeev Joshi²

`rajeev.joshi@jpl.nasa.gov`

Klaus Havelund²

`klaus.havelund@jpl.nasa.gov`

¹David Cheriton School of Computer Science, University of Waterloo, Canada

²Jet Propulsion Laboratory, California Inst. of Technology, USA



UNIVERSITY OF WATERLOO
FACULTY OF MATHEMATICS
David R. Cheriton School
of Computer Science



Jet Propulsion Laboratory
California Institute of Technology

A Language For Understanding What Space Robots Are Doing

- 1 The Evolution of RV logics
- 2 Telemetry Streams and Problem Definition
- 3 The `nfer` Notation
- 4 The `nfer` Scala DSL
- 5 Conclusion

The Evolution of Runtime Verification Logics

■ $m_{prop} \in \Sigma^* \rightarrow \mathbb{B}$

The Evolution of Runtime Verification Logics

- $m_{prop} \in \Sigma^* \rightarrow \mathbb{B}$

- $m_{data} \in (\Sigma \times \mathcal{V}^*)^* \rightarrow \mathbb{B}$

The Evolution of Runtime Verification Logics

- $m_{prop} \in \Sigma^* \rightarrow \mathbb{B}$

- $m_{data} \in (\Sigma \times \mathcal{V}^*)^* \rightarrow \mathbb{B}$

- $m_{comp} \in (\Sigma \times \mathcal{V}^*)^* \rightarrow \mathcal{V}$

Property `HasNext`

This property applies to every `java.util.Iterator` object, and requires that `hasNext()` be called before `next()` and that `hasNext()` returns true. Two events are monitored:

- `hasNext(i, r)` is triggered when `hasNext()` is called on Iterator *i* with result *r*.
- `next(i)` is triggered when `next()` is called on Iterator *i*.

Property Velocity

The speed of a monitored object should never go beyond a certain value v_{max} . The object moves along a straight line, and its distance from point 0 and the current time is repeatedly sampled. One event is monitored:

- $\text{pos}(d, t)$ indicates the current distance d from starting point 0, and the current time t .

Property MaxWithdrawal

The following property is rooted in the domain of fraud detection. It requires that, at each point in time, the sum of withdrawals of each user in the last 30 days does not exceed the limit of 10,000. One event is monitored:

- $\text{withdraw}(u, a, t)$ captures that the user u has withdrawn the amount a at time t .

The HasNext Property (Propositional Case)

$$\Box (\text{next} \rightarrow \bullet(\neg \text{next } S \text{ hasNext}))$$

$$\phi ::=$$

$$\begin{array}{l}
 r(t_1, \dots, t_{\ell(r)}) \mid \neg\phi \mid \phi_1 \vee \phi_2 \mid \exists x. \phi \\
 \mid \bullet_I \phi \mid \phi_1 S_I \phi_2 \mid \bigcirc_I \phi \mid \phi_1 U_I \phi_2 \\
 \mid [\omega_t \bar{z}. \phi](y; \bar{g})
 \end{array}$$

$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models p(t_1, \dots, t_{\ell(r)})$	iff	$(v(t_1), \dots, v(t_{\ell(r)})) \in p^{\mathcal{D}_i}$
$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \neg\psi$	iff	$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \not\models \psi$
$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \psi \vee \psi'$	iff	$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \psi$ or $(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \psi'$
$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \exists x. \psi$	iff	$(\bar{\mathcal{D}}, \bar{\tau}, v[x \mapsto d], i) \models \psi$, for some $d \in \mathbb{D}$
$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \bullet_I \psi$	iff	$i > 0$, $\tau_i - \tau_{i-1} \in I$, and $(\bar{\mathcal{D}}, \bar{\tau}, v, i-1) \models \psi$
$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \circ_I \psi$	iff	$\tau_{i+1} - \tau_i \in I$, and $(\bar{\mathcal{D}}, \bar{\tau}, v, i+1) \models \psi$
$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \psi S_I \psi'$	iff	for some $j \leq i$, $\tau_i - \tau_j \in I$, $(\bar{\mathcal{D}}, \bar{\tau}, v, j) \models \psi'$, and $(\bar{\mathcal{D}}, \bar{\tau}, v, k) \models \psi$, for all k with $j < k \leq i$
$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models \psi U_I \psi'$	iff	for some $j \geq i$, $\tau_j - \tau_i \in I$, $(\bar{\mathcal{D}}, \bar{\tau}, v, j) \models \psi'$, and $(\bar{\mathcal{D}}, \bar{\tau}, v, k) \models \psi$, for all k with $i \leq j < k$

$$(\bar{\mathcal{D}}, \bar{\tau}, v, i) \models [\omega_t \bar{z}. \psi](y; \bar{g}) \quad \text{iff} \quad v(y) = \omega(M)$$

where $M : \mathbb{D} \rightarrow \mathbb{N} \cup \{\infty\}$ is the multi-set

$$\{v[\bar{z} \mapsto \bar{d}](t) \mid (\bar{\mathcal{D}}, \bar{\tau}, v[\bar{z} \mapsto \bar{d}], i) \models \psi, \text{ for some } \bar{d} \in \mathbb{D}^{|\bar{z}|}\}.$$

The HasNext Property in MonPoly

$$\Box \forall i. (\text{next}(i) \rightarrow \bullet(\neg \text{next}(i) \text{ S hasNext}(i, \text{true})))$$

The Velocity Property in MonPoly

$$\Box \forall d, t. (\text{pos}(d, t) \rightarrow \bigcirc \exists d', t'. \text{pos}(d', t') \wedge d' - d \leq v_{\max} \cdot (t' - t))$$

The MaxWithdrawal Property in MonPoly

$\square \forall s, u.$

$([\text{SUM}_a a, t. \blacklozenge_{[0,30)} \text{withdraw}(u, a) \wedge \text{ts}(t)](s; u) \rightarrow s < 10000)$

Rule-based RV with LogFire

```
class HasNext extends Monitor {  
  val hasNext, next = event  
  val Safe = fact  
  
  "r1" -- hasNext('i, true)  $\mapsto$  Safe('i)  
  "r2" -- Safe('i) & next(' i)  $\mapsto$  remove(Safe)  
  "r3" -- next('i) & not(Safe(' i))  $\mapsto$  fail()  
}
```

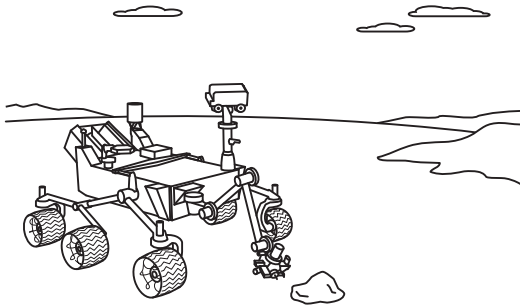
The Telemetry Stream

Human and machine comprehension of deployed spacecraft systems must be accomplished by the parsing of **telemetry**.

The Telemetry Stream

Human and machine comprehension of deployed spacecraft systems must be accomplished by the parsing of **telemetry**.

The spacecraft in question is a rover on Mars.



The Telemetry Stream



1

```
rh = open("radio", O_RDWR, 0600);  
if (rh == NULL)  
    return ERROR;  
  
status = ioctl (rh, IOBAUDRATE, 9600);  
  
report_evr(START_RADIO, S_CODE, status);  
  
write(rh, &buffer, n_bytes);
```

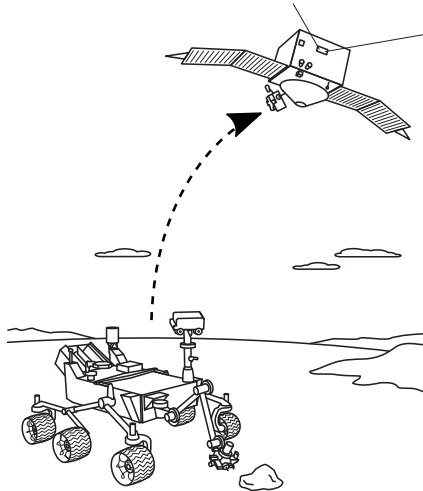
Telemetry messages (EVRs):

```
report_evr(START_RADIO, S_CODE, status);
```

- 1 Work like printf calls
- 2 Flat scheme
- 3 Added during software development

The Telemetry Stream

EVRs are transmitted in batches during orbiter overflights.



The Telemetry Stream

Label	Time	Parameters
DOWNLINK	10	size → 430
BOOT_S	42	count → 3
TURN_ANTENNA	80	
START_RADIO	90	baud → 9600
DOWNLINK	100	size → 420
BOOT_E	160	
STOP_RADIO	205	
BOOT_S	255	count → 4
START_RADIO	286	
BOOT_E	312	
TURN_ANTENNA	412	



vrf1
.py

tst2
.awk

chk3
.py

Ad-hoc verification scripts:

- 1 Scripts are hard to maintain
- 2 Each script must parse all events
- 3 Definitions may not be consistent between scripts
- 4 Scripts are specific to teams

We want to replace these scripts with a better system

- 1 Written in a consistent language
- 2 Each abstraction is defined once
- 3 Cross functional specifications are easier

Our Requirements

Label	Time	Parameters
DOWNLINK	10	size → 430
BOOT_S	42	count → 3
TURN_ANTENNA	80	
START_RADIO	90	baud → 9600
DOWNLINK	100	size → 420
BOOT_E	160	
STOP_RADIO	205	
BOOT_S	255	count → 4
START_RADIO	286	
BOOT_E	312	
TURN_ANTENNA	412	

- 1 A BOOT interval starts with a BOOT_S (boot start) event and ends with a BOOT_E (boot end) event.
- 2 A DBOOT (double boot) interval consists of two consecutive BOOT intervals, with no more than 5-minutes from the start of the first BOOT interval to the end of the second BOOT interval.
- 3 A RISK interval is a DBOOT interval during which a DOWNLINK occurs.

Our Requirements

Label	Time	Parameters
BOOT_S	42	count → 3
TURN_ANTENNA	80	
START_RADIO	90	baud → 9600
DOWNLINK	100	size → 420
BOOT_E	160	
BOOT_S	255	count → 4
START_RADIO	286	
BOOT_E	312	

Our Requirements

Label	Time	Parameters
BOOT_S	42	count $\rightarrow$ 3
TURN_ANTENNA	80	
START_RADIO	90	baud $\rightarrow$ 9600
* DOWNLINK	100	size $\rightarrow$ 420
BOOT_E	160	
BOOT_S	255	count $\rightarrow$ 4
START_RADIO	286	
BOOT_E	312	

Visualization is an Objective

NAME	TIME	PARAMS		
DOWNLINK	10	size -> 430		
BOOT_S	42	count -> 3		
TURN_ANTENNA	80			
START_RADIO	90		<i>BOOT</i>	
DOWNLINK	100	size -> 420		
BOOT_E	160		<i>DBOOT</i>	<i>RISK</i>
STOP_RADIO	205			
BOOT_S	255	count -> 4		
START_RADIO	286		<i>BOOT</i>	
BOOT_E	312			
TURN_ANTENNA	412			

- 1 Define intervals (start and end)
- 2 Refer to time stamps
- 3 Refer to data

Preliminary Definitions

types

$\mathbb{C} = \mathbb{N}$	<i>// time stamps</i>
$\mathbb{M} = \text{Id} \xrightarrow{m} \mathbb{V}$	<i>// maps</i>
$\mathbb{E} = \text{Id} \times \mathbb{C} \times \mathbb{M}$	<i>// Event</i>
$\mathbb{T} = \mathbb{E}^*$	<i>// Trace</i>

Preliminary Definitions

types

$\mathbb{C} = \mathbb{N}$ *// time stamps*

$\mathbb{M} = \text{Id} \xrightarrow{m} \mathbb{V}$ *// maps*

$\mathbb{E} = \text{Id} \times \mathbb{C} \times \mathbb{M}$ *// Event*

$\mathbb{T} = \mathbb{E}^*$ *// Trace*

$\mathbb{I} = \text{Id} \times \mathbb{C} \times \mathbb{C} \times \mathbb{M}$ *// intervals*

$\mathbb{P} = 2^{\mathbb{I}}$ *// pools*

Preliminary Definitions

types

$\mathbb{C} = \mathbb{N}$ // time stamps

$\mathbb{M} = \text{Id} \xrightarrow{m} \mathbb{V}$ // maps

$\mathbb{E} = \text{Id} \times \mathbb{C} \times \mathbb{M}$ // Event

$\mathbb{T} = \mathbb{E}^*$ // Trace

$\mathbb{I} = \text{Id} \times \mathbb{C} \times \mathbb{C} \times \mathbb{M}$ // intervals

$\mathbb{P} = 2^{\mathbb{I}}$ // pools

$(\text{id}, t, M) \in \mathbb{E}$

$\text{id}(t, M)$

$M_{\perp} = M \cup \{\perp\}$

Conversion of Trace to a Pool

We convert the trace to a set of intervals - a *pool*.

$$init: \mathbb{T} \rightarrow \mathbb{P}$$

$$init(\tau) = \{ (\eta, t, t, M) \mid \eta(t, M) \in \tau \}$$

An nfer specification consists of a list of *labeling* rules, each of the form:

$$\eta \leftarrow \eta_1 \oplus \eta_2 \text{ \textbf{map} } \Phi$$

where:

$$\eta, \eta_1, \eta_2 : \text{Id}$$

$$\oplus : \mathbb{C}^6 \rightarrow \text{Bool}$$

$$\Phi : \mathbb{M} \times \mathbb{M} \rightarrow \mathbb{M}_\perp$$

Let Δ be the type of rules.

A rule takes a pool as input and returns a pool of new intervals.

Specifications are sets of rules, where the specification takes a pool as input and returns the union of the returned pools from the rules with the original pool.

The final result is found by iterating to a fixed point.

One Step Semantics of Rule Applied to Pool

$$R \llbracket _ \rrbracket : \Delta \rightarrow \mathbb{P} \rightarrow \mathbb{P}$$

$$\begin{aligned} R \llbracket \eta \leftarrow \eta_1 \oplus \eta_2 \textbf{map } \Phi \rrbracket \pi = \\ \{ (\eta, s, e, M) \in \mathbb{I} \mid \\ \quad \exists s_1, e_1, s_2, e_2 \in \mathbb{C} \bullet \exists J, K \in \mathbb{M} \bullet \\ \quad (\eta_1, s_1, e_1, J) \in \pi \wedge \\ \quad (\eta_2, s_2, e_2, K) \in \pi \wedge \\ \quad \oplus(s_1, e_1, s_2, e_2, s, e) \wedge \\ \quad M = \Phi(J, K) \neq \perp \\ \} \end{aligned}$$

Types

Rules : Δ , Pools : $\mathbb{P}$, Intervals : $\mathbb{I}$, Timestamps : $\mathbb{C}$, Maps : $\mathbb{M}$

One Step Semantics of Spec. Applied to Pool

$$S \llbracket _ \rrbracket : \Delta^* \rightarrow \mathbb{P} \rightarrow \mathbb{P}$$

$$S \llbracket \delta_1 \dots \delta_n \rrbracket \pi = \pi \cup R \llbracket \delta_1 \rrbracket \pi \cup \dots \cup R \llbracket \delta_n \rrbracket \pi$$

Types

Rules : Δ , Pools : $\mathbb{P}$

Semantics of Spec. Applied to Trace

$$T \llbracket _ \rrbracket : \Delta^* \rightarrow \mathbb{T} \rightarrow \mathbb{P}$$

$$\begin{aligned} T \llbracket \delta_1 \dots \delta_n \rrbracket \tau = \\ \text{least } \pi \in \mathbb{P} \text{ such that} \\ \text{init}(\tau) \subseteq \pi \\ \wedge \\ \pi = S \llbracket \delta_1 \dots \delta_n \rrbracket (\pi) \end{aligned}$$

Types

Rules : Δ , Traces: $\mathbb{T}$, Pools : $\mathbb{P}$

Example:

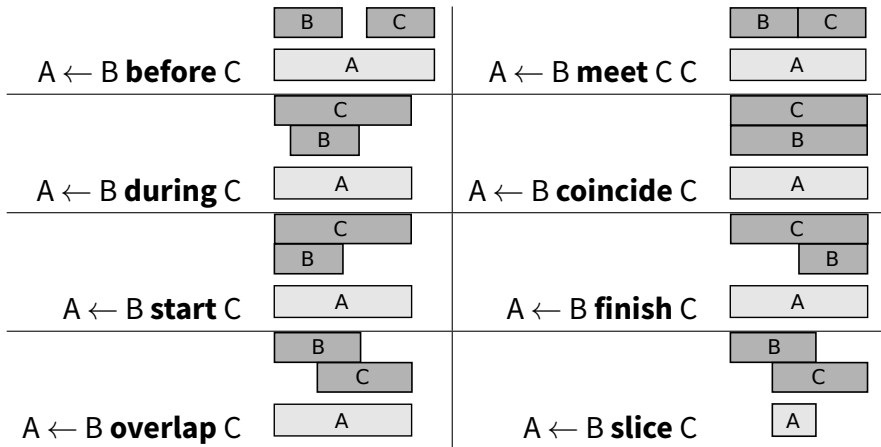
$$\text{BOOT} \leftarrow \text{BOOT_S} \oplus \text{BOOT_E} \mathbf{map} \Phi$$

where the two functions $\oplus$ and Φ are defined as follows:

$$\begin{aligned} \oplus(s_1, e_1, s_2, e_2, s, e) &= e_1 < s_2 \wedge s = s_1 \wedge e = e_2 \\ \Phi(m_1, m_2) &= [count \mapsto m_1(count)] \end{aligned}$$

Predefined Time Constraint Operators

Derived forms based on Allen Logic (Not exactly)



Predefined Time Constraint Operators

Operator $\oplus$	$\oplus(s_1, e_1, s_2, e_2, s, e)$
before	$e_1 < s_2 \wedge s = s_1 \wedge e = e_2$
meet	$e_1 = s_2 \wedge s = s_1 \wedge e = e_2$
during	$s_1 \geq s_2 \wedge e_1 \leq e_2 \wedge s = s_2 \wedge e = e_2$
coincide	$s = s_1 = s_2 \wedge e = e_1 = e_2$
start	$s = s_1 = s_2 \wedge e = \max(e_1, e_2)$
finish	$s = \min(s_1, s_2) \wedge e = e_1 = e_2$
overlap	$s_1 < e_2 \wedge s_2 < e_1 \wedge s = \min(s_1, s_2) \wedge e = \max(e_1, e_2)$
slice	$s_1 < e_2 \wedge s_2 < e_1 \wedge s = \max(s_1, s_2) \wedge e = \min(e_1, e_2)$

$\text{BOOT} \leftarrow \text{BOOT_S} \textbf{ before } \text{BOOT_E}$
 $\textbf{map} (\lambda m_1, m_2 . [\text{count} \mapsto m_1(\text{count})])$

$\text{DBOOT} \leftarrow \text{BOOT} \textbf{ before } \text{BOOT}$
 $\textbf{within} (\lambda s_1, e_1, s_2, e_2, s, e . e - s \leq 300)$
 $\textbf{map} (\lambda m_1, m_2 . m_2)$

$\text{RISK} \leftarrow \text{DOWNLINK} \textbf{ during } \text{DBOOT}$
 $\textbf{map} (\lambda m_1, m_2 . m_2)$

Abbreviation Used in Above Example

Let $\odot \in$

$\{\text{before, meet, during, coincide, start, finish, overlap, slice}\}$,
and let $\odot_p$ denote the corresponding clock predicate.

The following abbreviation is introduced:

$$\eta \leftarrow \eta_1 \odot \eta_2 \text{ within } \Theta \text{ map } \Phi$$

where $\Theta : \mathbb{C}^6 \rightarrow \mathbb{B}$ is a predicate on six time stamps. This is synonymous with:

$$\eta \leftarrow \eta_1 (\odot_p \wedge \Theta) \eta_2 \text{ map } \Phi$$

Default Values for Additional Time Constraints and Map Functions

We shall allow time constraints (**within**) and map transformations (**map**) to be left out in rules, in which case they assume the default function values respectively:

- $\lambda s_1, e_1, s_2, e_2, s, e. \text{true}$
- $\lambda m_1, m_2. []$

Label	Start	End	Parameters
BOOT_S	42	42	count $\rightarrow$ 3
DOWNLINK	100	100	size $\rightarrow$ 420
BOOT_E	160	160	
BOOT_S	255	255	count $\rightarrow$ 4
BOOT_E	312	312	

1 : { (BOOT, 42, 160, [count $\mapsto$ 3]),
 (BOOT, 255, 312, [count $\mapsto$ 4]) }

Label	Start	End	Parameters
BOOT_S	42	42	count $\rightarrow$ 3
DOWNLINK	100	100	size $\rightarrow$ 420
BOOT_E	160	160	
BOOT_S	255	255	count $\rightarrow$ 4
BOOT_E	312	312	
BOOT	42	160	count $\rightarrow$ 3
BOOT	255	312	count $\rightarrow$ 4

2 : { (DBOOT, 42, 312, [count $\mapsto$ 4]) }

Label	Start	End	Parameters
BOOT_S	42	42	count $\rightarrow$ 3
DOWNLINK	100	100	size $\rightarrow$ 420
BOOT_E	160	160	
BOOT_S	255	255	count $\rightarrow$ 4
BOOT_E	312	312	
BOOT	42	160	count $\rightarrow$ 3
BOOT	255	312	count $\rightarrow$ 4
DBOOT	42	312	count $\rightarrow$ 4

3 : { (RISK, 42, 312, [$count \mapsto 4$]) }

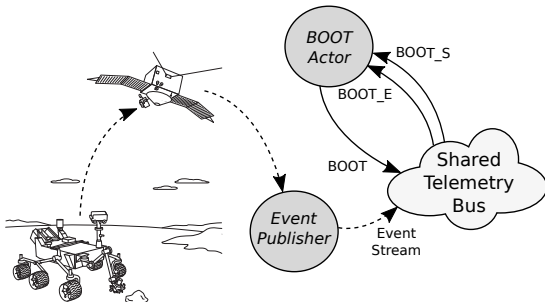
Label	Start	End	Parameters
BOOT_S	42	42	count $\rightarrow$ 3
DOWNLINK	100	100	size $\rightarrow$ 420
BOOT_E	160	160	
BOOT_S	255	255	count $\rightarrow$ 4
BOOT_E	312	312	
BOOT	42	160	count $\rightarrow$ 3
BOOT	255	312	count $\rightarrow$ 4
DBOOT	42	312	count $\rightarrow$ 4
RISK	42	312	count $\rightarrow$ 4

Example in the Scala DSL

```
class DoubleBoot extends Nfer {  
  "BOOT" :- ("BOOT_S" before "BOOT_E" map {  
    case (m1,m2)  $\Rightarrow$  Map("count"  $\rightarrow$  m1("count"))  
  })  
  
  "DBOOT" :- ("BOOT" before "BOOT" within 300 map (_.2))  
  
  "RISK" :- ("DOWNLINK" during "DBOOT" map (_.2))  
}
```

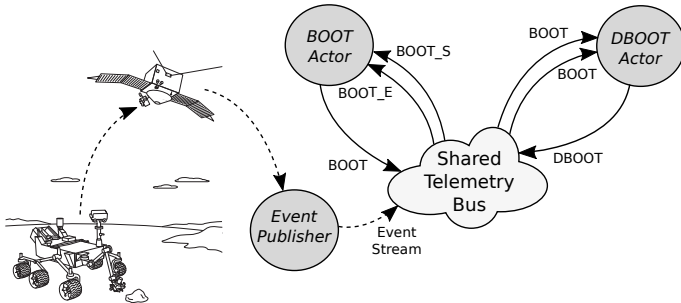
Implementation using Actors

```
"BOOT" :- ("BOOT_S" before "BOOT_E" map {  
  case (m1,m2) => Map("count" -> m1("count"))  
})
```



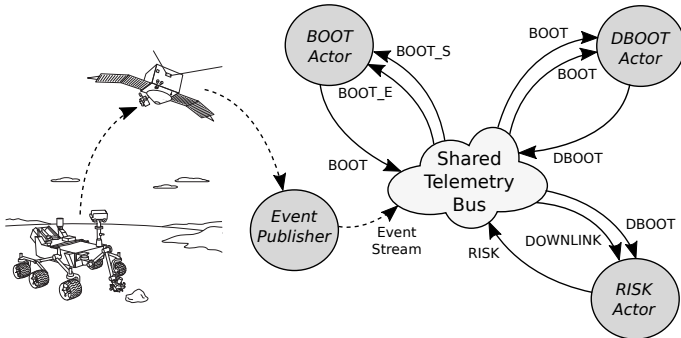
Implementation using Actors

"DBOOT" :- ("BOOT" before "BOOT" within 300 map (.__2))



Implementation using Actors

"RISK" :— ("DOWNLINK" during "DBOOT" map (_._2))



The following rule:

```
"BOOT" :- ("BOOT_S" before "BOOT_E" map {  
    case (m1,m2)  $\Rightarrow$  Map("count"  $\rightarrow$  m1("count"))  
})
```


The following rule:

```
"BOOT" :- ("BOOT_S" before "BOOT_E" map {  
  case (m1,m2)  $\Rightarrow$  Map("count"  $\rightarrow$  m1("count"))  
})
```

Is equivalent to the following function call(s):

```
liftRuleName("BOOT").:-(  
  ( liftOperand ("BOOT_S").before(liftOperand ("BOOT_E"))).map {  
    case (m1,m2)  $\Rightarrow$  Map("count"  $\rightarrow$  m1("count"))  
  )
```

How does this Work?

```
implicit def liftRuleName(s: String) = new {  
  def :-(op: Op) = makeRules(s, op)  
}
```

How does this Work?

```
implicit def liftRuleName(s: String) = new {  
  def :-(op: Op) = makeRules(s, op)  
}
```

```
implicit def liftOperand(s: String) = new Op(s)
```

```
case class Op(s : String, left : Op = null, right : Op = null,  
             op: Fun = null)  
{  
  def before(e: Op) = Op(..., this, e, BEFORE)  
  def during(e: Op) = Op(..., this, e, DURING)  
  ...  
}
```

How does this Work?

```
def BEFORE(...) {  
  makeRule(...,  
    { case ( i1: Interval , i2: Interval )  $\Rightarrow$  ( i1 .end < i2 . start )},  
    { case ( i1: Interval , i2: Interval )  $\Rightarrow$  ( i1 . start , i2 .end) })  
}
```

A More Realistic Example

Requirement

Label warning events corresponding to known harmless scenarios.

```
"cmdExec" :- ("CMD_DISPATCH" before "CMD_COMPLETE" map {  
  case (m1, m2) =>  
    if (m1("cmd") == m2("cmd"))  
      Some(Map("cmd" → m1("cmd")))  
    else  
      None  
})  
  
"telecom0208" :- ("TLM_TR_ERROR" during "cmdExec" map {  
  case (_, m2) =>  
    if ((m2("cmd") == "MOB_PRM") || (m2("cmd") == "ARM_PRM"))  
      Some(Map())  
    else  
      None  
})
```

Innocent Command Failure

```
def delay15(s1,e1,s2,e2,s,e : Double): Boolean = e - s <= 15
```

```
"instCmdFail" :- ("INST_PWR_ON" before  
  ("INST_CMD_FAIL" before "INST_RECOVER") within delay15)
```

Innocent Command Failure

def delay15(s1,e1,s2,e2,s,e : Double): Boolean = e - s <= 15

"instCmdFail" :- ("INST_PWR_ON" before
("INST_CMD_FAIL" before "INST_RECOVER") within delay15)

"instCmdFail" :- ("INST_PWR_ON" before
("INST_CMD_FAIL" before "INST_RECOVER") within 15)

Innocent Command Failure

```
def delay15(s1,e1,s2,e2,s,e : Double): Boolean = e - s <= 15
```

```
"instCmdFail" :- ("INST_PWR_ON" before  
  ("INST_CMD_FAIL" before "INST_RECOVER") within delay15)
```

```
"instCmdFail" :- ("INST_PWR_ON" before  
  ("INST_CMD_FAIL" before "INST_RECOVER") within 15)
```

```
implicit def liftTime (t: Int): (Int, Int, Int, Int, Int, Int)  $\Rightarrow$  Boolean =  
  {case (s1,e1,s2,e2,s,e)  $\Rightarrow$  e - s <= t
```

Innocent Task Starvation

```
"comm" :- ("WIN_BEGIN" before "WIN_END" map {  
    case (m1, __) => Map("wid" -> m1("wid"))  
})
```

```
"vdp" :- ("VDP_START" before "VDP_STOP")
```

```
"starvationOk" :- "TASK_STARVATION" during ("vdp" slice "comm")
```

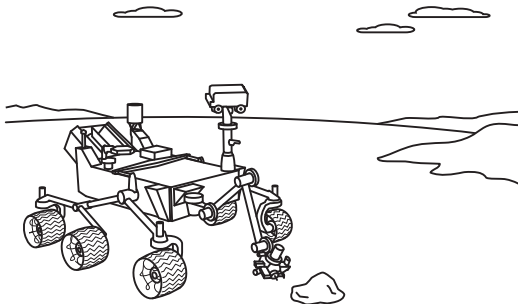
Conclusion

- 1 Progression of RV logics (prop,data,comp)
- 2 Where is the line between logic (DSL) and a programming language?

Future work:

- 1 Support for out-of-order events
- 2 Checking properties (use in traditional RV)
- 3 Specification mining

Thanks!



¹Image by <https://www.vecteezy.com/free-vector/icons>